

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
ДЕПАРТАМЕНТ ЦИФРОВЫХ, РОБОТОТЕХНИЧЕСКИХ СИСТЕМ И
ЭЛЕКТРОНИКИ

КУРСОВАЯ РАБОТА

по дисциплине

«ИНТЕРНЕТ-ТЕХНОЛОГИИ»

на тему:

«Разработка веб-приложения для управления задачами»

Выполнил:

Милосердов Илья Яковлевич

Студент 4 курса группы ПИН-б-з-22-1

Направления подготовки

09.03.03 Прикладная информатика

заочной формы обучения

(подпись)

Руководитель работы:

(ФИО, должность, кафедра)

Работа допущена к защите _____
(подпись руководителя) (дата)

Работа выполнена и
защищена с оценкой _____ Дата защиты _____

Члены комиссии: _____
(должность) (подпись) (И.О. Фамилия)

Ставрополь, 2026 г.

УТВЕРЖДАЮ
Старший преподаватель департамента цифровых,
робототехнических систем
и электроники
Щеголев А.А.

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники
Направление подготовки 09.03.03 Прикладная информатика
Направленность (профиль) Прикладная информатика в экономике

ЗАДАНИЕ

на курсовую работу

студента Милосердова Ильи Яковлевича

(фамилия, имя, отчество)

по дисциплине Интернет-технологии

1.Тема работы «Разработка веб-приложения для управления задачами»

2.Цель разработать полноценное веб-приложение для управления задачами, включающее клиентскую и серверную части, систему аутентификации, инструменты для создания, распределения и отслеживания задач.

3.Технологический стек Python, Django, Django REST Framework, React, TypeScript, Docker.

4.Перечень подлежащих разработке вопросов:

а) по теоретической части Анализ предметной области управления задачами, обзор существующих решений, обоснование выбора технологического стека, принципы построения SPA-приложений и REST API.

б) по практической части проектирование моделей данных (Task, User), реализация backend на Django REST Framework с JWT-аутентификацией, разработка frontend на React с TypeScript, реализация Kanban-доски с drag-and-drop, поддержка тёмной и светлой тем, контейнеризация с Docker и развёртывание на VPS.

5.Исходные данные:

техническое задание на разработку веб-приложения для управления задачами, документация по Django, Django REST Framework, React, TypeScript, Tailwind CSS, Docker.

6. Список рекомендуемой литературы: официальная документация Django, Django REST Framework, React, TypeScript, Tailwind CSS, Docker.

7.Контрольные сроки представления отдельных разделов курсовой работы:

25 % - _____ “ ” _____ 20_ г.
50 % - _____ “ ” _____ 20_ г.
75 % - _____ “ ” _____ 20_ г.
100 % - _____ “ ” _____ 20_ г.

8. Срок защиты студентом курсовой работы “ ” _____ 20_ г.

Дата выдачи задания “ ” _____ 20_ г.

Руководитель курсовой работы

(ученая степень, звание) (личная подпись) (инициалы, фамилия)

Задание принял(а) к исполнению студент(ка) _____ формы обучения
_____ курса _____ группы _____

(личная подпись)

(инициалы, фамилия)

Отзыв

на курсовой проект студента Милосердова Ильи Яковлевича
Тема работы: «Разработка веб-приложения для управления задачами».

Курсовая работа посвящена актуальной теме разработки систем управления задачами. Продемонстрированы хорошие навыки проектирования и разработки веб-приложений.

В ходе выполнения работы были использованы современные технологии:

- Backend: Django и Django REST Framework.
- Frontend: React и экосистема (Vite, TailwindCSS).
- Контейнеризация: Docker и Docker Compose.

Работа выполнена на высоком техническом уровне. Реализованный функционал соответствует техническому заданию. Приложение имеет продуманную архитектуру (клиент-серверная) и код, соответствующий стандартам разработки.

Положительные стороны:

- Использование разделения ответственности (Frontend/Backend).
- Настройка Docker окружения для быстрого развертывания.
- Реализация JWT авторизации и REST API.
- Поддержка Kanban-доски с drag-and-drop.

Недостатки: Существенных недостатков не выявлено.

Заключение: Студент заслуживает оценки «_____».

Научный руководитель: Старший преподаватель / Щеголев А.А.

Дата: _____

ОГЛАВЛЕНИЕ

Введение	6
1 Теоретическая часть	8
1.1 Анализ предметной области управления задачами	8
1.2 Обзор существующих решений	8
1.3 Обоснование выбора технологий разработки	9
1.4 Принципы построения SPA-приложений и REST API	10
1.5 Паттерны проектирования веб-приложений	10
1.6 Безопасность веб-приложений и защита данных	11
1.7 Контейнеризация и современные практики развёртывания	13
1.8 Анализ требований к программному обеспечению	14
1.9 Методология разработки и планирование проекта	15
2 Практическая часть	16
2.1 Постановка задачи и системные требования	16
2.2 Обоснование выбора стека технологий	17
2.3 Проектирование архитектуры и базы данных	18
2.4 Разработка серверной части (Backend)	20
2.5 Разработка клиентской части и вёрстки (Frontend)	21
2.6 Автоматизация процессов разработки и развёртывания (CI/CD)	23
2.7 Тестирование и верификация работоспособности	25
2.8 Описание пользовательского интерфейса	26
2.9 Руководство пользователя	28
2.10 Оценка качества и перспективы развития	30
2.11 Документирование API и примеры использования	31
2.12 Безопасность и защита данных в приложении	33
Заключение	37
Список использованных источников	39

Введение

В современном мире эффективное управление задачами является важной составляющей как личной, так и командной работы. Рост объёмов информации, необходимость координации действий нескольких исполнителей и требование соблюдения сроков делают ручное планирование неэффективным. Записи в блокнотах, электронные таблицы и мессенджеры не обеспечивают централизованного хранения статусов задач, приоритетов и сроков их выполнения.

Веб-приложения для управления задачами позволяют систематизировать рабочий процесс, предоставлять наглядное представление о текущем состоянии дел, распределять ответственность между участниками и контролировать соблюдение дедлайнов. Такие системы находят применение в IT-командах, образовательных учреждениях, проектном менеджменте и личном планировании.

Объектом исследования является процесс разработки клиент-серверных веб-приложений с использованием современных фреймворков и технологий контейнеризации.

Предметом исследования выступают методы и средства создания веб-приложения для управления задачами, включая проектирование REST API, реализацию одностороннего интерфейса (SPA), организацию аутентификации на основе JWT, поддержку тем оформления, а также контейнеризацию приложения с использованием Docker.

Целью курсовой работы является разработка полноценного веб-приложения для управления задачами, включающего клиентскую и серверную части, систему аутентификации, инструменты для создания, редактирования, распределения и отслеживания задач.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и обзор существующих решений для управления задачами;
- сформулировать функциональные и нефункциональные требования к разрабатываемому приложению;
- спроектировать архитектуру системы на основе разделения ответственности между backend и frontend;
- спроектировать модели данных (Task, User) и REST API;
- реализовать backend-часть на языке Python с использованием Django и Django REST Framework;
- реализовать frontend-часть на языке TypeScript с использованием React и Vite;
- реализовать Kanban-доску с drag-and-drop для перемещения задач между статусами;
- реализовать поддержку тёмной и светлой тем оформления;
- выполнить контейнеризацию приложения с использованием Docker и Docker Compose;
- провести функциональное тестирование основных сценариев работы приложения.

Практическая значимость работы заключается в том, что разработанное приложение может быть использовано для управления личными и командными задачами, а также в качестве учебного примера построения современных клиент-серверных веб-приложений.

Курсовая работа состоит из введения, теоретической и практической частей, заключения и списка использованных источников. В теоретической части рассматриваются предметная область управления задачами, существующие решения и технологический стек. В практической части

описываются требования, архитектура, разработка backend и frontend, развёртывание и тестирование приложения.

1 Теоретическая часть

1.1 Анализ предметной области управления задачами

Управление задачами — это процесс планирования, организации и контроля выполнения работ. В рамках небольших проектов и личного планирования часто используются простые инструменты: бумажные записи, электронные таблицы, заметки в мессенджерах. Однако при увеличении количества задач и участников такие подходы становятся неэффективными: информация теряется, статусы задач не отслеживаются, сроки выполнения нарушаются.

Разрабатываемое приложение должно предоставлять пользователю удобный интерфейс для создания задач, установки приоритетов, назначения сроков выполнения, распределения задач между исполнителями и визуализации прогресса. В качестве основного способа визуализации выбрана Kanban-доска, позволяющая перемещать задачи между колонками статусов.

1.2 Обзор существующих решений

На рынке представлен широкий спектр решений для управления задачами. Среди наиболее известных можно выделить следующие.

Trello — популярный сервис для управления задачами и проектами на основе Kanban-досок. Обладает простым интерфейсом, поддерживает совместную работу и интеграции. Является коммерческим облачным сервисом.

Jira — профессиональная система для управления проектами и задачами, широко используемая в IT-компаниях. Поддерживает гибкие методологии, но имеет высокую сложность в освоении и платную лицензию.

Asana — инструмент для управления задачами и командной работы. Предоставляет списки, доски, календари и диаграммы Ганта. Распространяется по подписной модели.

Анализ существующих решений показывает, что большинство из них являются облачными сервисами с ограниченной бесплатной версией и не предоставляют возможности самостоятельного развёртывания. Это обуславливает целесообразность разработки собственного веб-приложения с открытым исходным кодом и возможностью развёртывания на собственном сервере.

1.3 Обоснование выбора технологий разработки

Для реализации веб-приложения выбран современный технологический стек, обеспечивающий высокую производительность, удобство разработки и возможность дальнейшего масштабирования.

В качестве языка программирования для backend-части выбран Python 3.10 — один из наиболее популярных языков программирования, отличающийся простым синтаксисом и широким выбором библиотек. Для реализации backend выбран фреймворк Django 5, предоставляющий встроенные средства для работы с базой данных (ORM), аутентификацией и администрированием.

Для реализации REST API поверх Django используется Django REST Framework, предоставляющий удобные инструменты для создания API, сериализаторов и аутентификации.

Клиентская часть разработана на языке TypeScript с использованием библиотеки React 18 и сборщика Vite. Для стилизации интерфейса используется Tailwind CSS, позволяющий быстро создавать адаптивные компоненты.

Контейнеризация приложения выполнена с помощью Docker и Docker Compose, что обеспечивает простоту развёртывания и переносимость между средами.

1.4 Принципы построения SPA-приложений и REST API

Одностраничное приложение (Single Page Application, SPA) — это веб-приложение, которое загружает один HTML-документ и динамически обновляет содержимое страницы при взаимодействии с пользователем. Такой подход позволяет создавать быстрые и отзывчивые интерфейсы, похожие на desktop-приложения.

В качестве подхода к организации взаимодействия между клиентом и сервером выбран архитектурный стиль REST (Representational State Transfer). REST API использует стандартные HTTP-методы: GET для получения данных, POST для создания, PATCH/PUT для обновления и DELETE для удаления. Данные передаются в формате JSON, что обеспечивает простоту обработки на различных платформах.

Использование SPA в сочетании с REST API позволяет разделить логику представления и бизнес-логику приложения. Сервер отвечает за хранение и обработку данных, а клиентская часть отвечает за отображение информации и взаимодействие с пользователем. Такое разделение упрощает разработку, тестирование и дальнейшее развитие приложения.

1.5 Паттерны проектирования веб-приложений

При проектировании веб-приложений широко применяются паттерны, позволяющие структурировать код, разделить ответственность между компонентами и упростить дальнейшую поддержку системы. Одним из наиболее известных паттернов является MVC (Model-View-Controller), в

котором приложение делится на три части: модель данных, представление и контроллер, обрабатывающий взаимодействие между ними.

Фреймворк Django использует близкую к MVC архитектуру MTV (Model-Template-View). В этой модели модель отвечает за данные и бизнес-логику, шаблон определяет внешний вид страницы, а представление обрабатывает запрос пользователя и возвращает сформированный ответ. Использование MTV позволяет чётко разделить ответственность между компонентами backend-части и упрощает тестирование.

Клиентская часть приложения построена на компонентном подходе, который является основой библиотеки React. Каждый компонент отвечает за отображение отдельного элемента интерфейса и может содержать собственное состояние и логику. Компоненты могут быть вложенными, переиспользуемыми и комбинироваться для построения сложных интерфейсов. Такой подход упрощает разработку, тестирование и дальнейшее расширение функционала.

Разделение ответственности между backend и frontend, а также использование паттернов проектирования позволяют создавать масштабируемые и поддерживаемые приложения. Серверная часть отвечает за бизнес-логику и хранение данных, а клиентская часть — за отображение информации и взаимодействие с пользователем. Такое разделение упрощает внесение изменений и позволяет развивать части приложения независимо друг от друга.

1.6 Безопасность веб-приложений и защита данных

Безопасность является одним из ключевых аспектов при разработке веб-приложений. Приложение должно защищать данные пользователей, предотвращать несанкционированный доступ и обеспечивать корректную обработку входных данных. Основными угрозами для веб-приложений

являются межсайтовый скриптинг (XSS), подделка межсайтовых запросов (CSRF), SQL-инъекции, а также несанкционированный доступ к защищённым ресурсам.

Для защиты API от несанкционированного доступа в приложении используется аутентификация на основе JWT-токенов. После успешной авторизации клиент получает access-токен, который передаётся в заголовке каждого запроса к защищённым ресурсам. Access-токен имеет ограниченное время жизни, что снижает риски при его компрометации. Для обновления access-токена используется refresh-токен, хранящийся в защищённом виде.

Django предоставляет встроенные механизмы защиты от CSRF-атак и SQL-инъекций. ORM Django использует параметризованные запросы, что исключает возможность SQL-инъекций при корректном использовании. Фреймворк также выполняет автоматическое экранирование данных при рендеринге шаблонов, что защищает от XSS-атак.

Клиентская часть приложения выполняет валидацию входных данных перед отправкой на сервер. Проверяются обязательные поля, корректность дат, допустимость приоритетов и статусов задач. Серверная часть дополнительно проверяет полученные данные с помощью сериализаторов Django REST Framework, что обеспечивает двухуровневую валидацию и повышает надёжность приложения.

При развёртывании приложения в production-окружении используется протокол HTTPS, который шифрует передаваемые данные и защищает их от перехвата. SSL-сертификаты выдаются бесплатным удостоверяющим центром Let's Encrypt и автоматически обновляются. Также настроены заголовки безопасности, ограничивающие выполнение встроенного кода и защищающие от ряда распространённых атак.

1.7 Контейнеризация и современные практики развёртывания

Контейнеризация является современным подходом к упаковке и развёртыванию приложений. Контейнер объединяет приложение и все его зависимости в единый образ, который может быть запущен на любой платформе, поддерживающей контейнерную среду выполнения. Это обеспечивает переносимость, изоляцию компонентов и воспроизводимость окружения.

В качестве инструмента контейнеризации используется Docker. Backend и frontend приложения упаковываются в отдельные Docker-образы, что позволяет развёртывать их независимо друг от друга. Для управления многоконтейнерной средой применяется Docker Compose, который описывает состав сервисов, их зависимости, сетевые настройки и подключаемые тома.

Использование Docker позволяет упростить настройку окружения разработки. Разработчику не требуется устанавливать Python, Node.js и другие зависимости вручную — достаточно запустить контейнеры с помощью одной команды. Это снижает вероятность ошибок, связанных с различием версий библиотек и операционных систем.

При развёртывании в production применяется обратный прокси-сервер Nginx, который обеспечивает раздачу статических файлов frontend, проксирование API-запросов к backend и терминацию SSL-соединений. Такая схема позволяет эффективно распределять нагрузку, кешировать статические ресурсы и централизованно управлять настройками безопасности.

Современные практики разработки также предполагают использование системы контроля версий Git и размещение исходного кода в удалённом репозитории. Это позволяет команде разработчиков совместно работать над проектом, отслеживать изменения, выполнять код-ревью и при необходимости откатываться к предыдущим версиям. В дальнейшем планируется настроить

непрерывную интеграцию и доставку (CI/CD), которая будет автоматически собирать образы, запускать тесты и развёртывать приложение при изменении кода.

1.8 Анализ требований к программному обеспечению

Перед началом разработки был проведён анализ требований к будущему приложению. Требования разделены на функциональные, описывающие поведение системы, и нефункциональные, определяющие качественные характеристики работы.

К функциональным требованиям относятся: управление учётными записями пользователей, создание и редактирование задач, назначение приоритетов и сроков, распределение задач между исполнителями, визуализация задач на Kanban-доске и в календаре, поддержка массовых операций и переключение тем оформления.

К нефункциональным требованиям относятся: клиент-серверная архитектура с REST API, использование JWT для аутентификации, адаптивный интерфейс, корректная работа в современных браузерах, возможность развёртывания в Docker-контейнерах, защита данных при передаче по сети, а также достаточная производительность при работе с несколькими сотнями задач.

Для наглядного представления взаимодействия пользователя с системой были выделены следующие роли и сценарии использования. Обычный пользователь может регистрироваться, входить в систему, создавать собственные задачи, редактировать их, перемещать между статусами и просматривать в календаре. Администратор в перспективе сможет управлять пользователями и настройками приложения. На этапе разработки основной акцент сделан на роли обычного пользователя.

1.9 Методология разработки и планирование проекта

Разработка приложения велась итерационным подходом с элементами гибкой методологии. Работа была разбита на несколько этапов: анализ и проектирование, разработка backend, разработка frontend, интеграция частей, тестирование и развёртывание. Каждый этап завершался проверкой результата и при необходимости корректировкой плана.

Для управления исходным кодом использовалась система контроля версий Git. Репозиторий размещён на GitHub, что позволяет сохранять историю изменений, создавать резервные копии и при необходимости возвращаться к предыдущим версиям. В процессе разработки применялся подход с небольшими коммитами, каждый из которых содержал логически завершённое изменение.

Планирование функционала выполнялось на основе технического задания и требований к курсовой работе. Приоритет отдавался реализации ключевых функций: аутентификации, управления задачами, Kanban-доски и развёртывания. Дополнительные возможности, такие как система уведомлений, расширенная фильтрация и ролевая модель, были отнесены к дальнейшему развитию проекта.

Такой подход позволил постепенно наращивать функционал приложения, своевременно выявлять ошибки и обеспечивать прозрачность процесса разработки. Использование современных инструментов и методологий способствовало получению структурированного и поддерживаемого программного продукта.

2 Практическая часть

2.1 Постановка задачи и системные требования

Целью практической части является создание веб-приложения для управления задачами, которое позволяет пользователям создавать, редактировать и удалять задачи, назначать им приоритеты и сроки выполнения, распределять задачи между исполнителями, а также отслеживать прогресс с помощью Kanban-доски и календаря.

Приложение должно предоставлять следующие возможности:

- регистрация и авторизация пользователей;
- создание, редактирование и удаление задач;
- установка приоритета и срока выполнения задачи;
- назначение исполнителя задачи;
- перемещение задач между статусами с помощью drag-and-drop;
- отображение задач в виде Kanban-доски;
- просмотр задач в календаре;
- поддержка тёмной и светлой тем оформления;
- массовое перемещение и удаление задач в одной колонке;
- развёртывание приложения в Docker-контейнерах.

К приложению предъявляются следующие системные требования:

- клиент-серверная архитектура с REST API;
- использование JWT-токенов для аутентификации;
- адаптивный интерфейс, корректно отображающийся на различных устройствах;
- возможность развёртывания на VPS-сервере с использованием Nginx;
- хранение данных в реляционной базе данных SQLite/PostgreSQL;
- защита маршрутов API от несанкционированного доступа.

Кроме функциональных требований, к приложению предъявляются нефункциональные требования, определяющие качество работы системы. К ним относятся удобство использования интерфейса, корректная обработка ошибок, достаточная производительность при работе с несколькими сотнями задач, а также простота развёртывания и обновления. Важным требованием является кроссплатформенность: приложение должно работать в современных браузерах на компьютерах и мобильных устройствах.

2.2 Обоснование выбора стека технологий

Выбор технологического стека обусловлен требованиями к производительности, удобству разработки и дальнейшему масштабированию приложения.

Python 3.10 выбран в качестве языка backend-разработки благодаря простому синтаксису, большому количеству готовых библиотек и широкой поддержке сообщества. Django 5 предоставляет готовые компоненты для работы с базой данных, аутентификации и администрирования, что ускоряет разработку.

Django REST Framework расширяет возможности Django для построения REST API, предоставляя сериализаторы, ViewSet'ы и систему разрешений. JWT-аутентификация реализуется с помощью библиотеки `django-rest-framework-simplejwt`.

React 18 выбран для frontend-разработки из-за компонентного подхода, высокой производительности и большой экосистемы. TypeScript обеспечивает статическую типизацию, что снижает количество ошибок на этапе разработки. Vite используется в качестве сборщика благодаря высокой скорости сборки и простоте конфигурации.

Tailwind CSS позволяет быстро создавать адаптивные интерфейсы с помощью утилитарных классов, что ускоряет вёрстку и упрощает поддержку стилей.

2.3 Проектирование архитектуры и базы данных

2.3.1 Проектирование архитектуры системы

Приложение построено по классической клиент-серверной архитектуре с разделением ответственности между backend и frontend. Серверная часть реализована на Django и Django REST Framework и предоставляет REST API для работы с данными. Клиентская часть разработана на React с использованием TypeScript и взаимодействует с сервером по HTTP.

Аутентификация пользователей выполняется с помощью JWT-токенов. После успешного входа клиент получает access-токен и refresh-токен, которые используются для доступа к защищённым ресурсам API. Access-токен имеет ограниченное время жизни, а refresh-токен позволяет получить новый access-токен без повторного ввода логина и пароля.

Такое разделение архитектуры обеспечивает независимость развития клиентской и серверной частей. Backend может быть использован различными клиентами, включая веб-приложение и мобильное приложение. Frontend, в свою очередь, может получать данные из API и не зависеть от конкретной реализации серверной части.

2.3.2 Проектирование базы данных

Основной сущностью приложения является задача (Task). Каждая задача содержит название, описание, статус, приоритет, дату создания, срок выполнения, исполнителя и создателя. Модель Task связана с моделью User внешними ключами assigned_to и created_by.

Схема базы данных включает таблицы:

- `auth_user` — стандартная таблица пользователей Django;
- `tasks_task` — таблица задач с полями `title`, `description`, `status`, `priority`, `created_at`, `due_date`, `assigned_to_id`, `created_by_id`.

Связь между таблицами `tasks_task` и `auth_user` реализована через внешние ключи. Поле `created_by` обязательно для заполнения и указывает на пользователя, создавшего задачу. Поле `assigned_to` может быть пустым, что соответствует задаче без назначенного исполнителя.

Ниже представлена реализация модели `Task` на языке Python:

Модель `Task` (backend/tasks/models.py)

```
class Task(models.Model):
    STATUS_CHOICES = [
        ('new', 'Новая'),
        ('in_progress', 'В работе'),
        ('done', 'Выполнена'),
    ]
    PRIORITY_CHOICES = [
        ('low', 'Низкий'),
        ('medium', 'Средний'),
        ('high', 'Высокий'),
    ]
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    status = models.CharField(max_length=20,
choices=STATUS_CHOICES, default='new')
    priority = models.CharField(max_length=20,
choices=PRIORITY_CHOICES, default='medium')
    created_at = models.DateTimeField(auto_now_add=True)
    due_date = models.DateTimeField(null=True, blank=True)
    assigned_to = models.ForeignKey(User,
on_delete=models.SET_NULL, null=True, blank=True,
related_name='tasks')
    created_by = models.ForeignKey(User,
on_delete=models.CASCADE, related_name='created_tasks')
```

2.4 Разработка серверной части (Backend)

2.4.1 Технологический стек backend

Серверная часть реализована на языке Python 3.10 с использованием фреймворка Django 5.0 и Django REST Framework. Django предоставляет готовые инструменты для работы с базой данных, аутентификацией, маршрутизацией и администрированием. Django REST Framework расширяет эти возможности для построения REST API.

Для аутентификации используется библиотека `django-rest-framework-simplejwt`, реализующая JWT-токены. Для обеспечения кросс-доменных запросов между frontend и backend используется `django-cors-headers`. В качестве базы данных используется SQLite в режиме разработки и PostgreSQL в production-окружении.

2.4.2 Модели, сериализаторы и маршрутизация

Маршрутизация API настроена в файле `config/urls.py`. Все endpoints API имеют префикс `/api/`. Аутентификация выполняется по адресам `/api/token/` и `/api/token/refresh/`. Задачи доступны по адресу `/api/tasks/`.

Маршрутизация API (backend/config/urls.py)

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('tasks.urls')),
    path('api/accounts/', include('accounts.urls')),
    path('api/token/', TokenObtainPairView.as_view(),
name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(),
name='token_refresh'),
]
```

Сериализатор `TaskSerializer` преобразует объекты модели `Task` в JSON и обратно. При создании задачи текущий пользователь автоматически

назначается создателем. Для отображения информации о создателе используется вложенный `UserSerializer`.

2.4.3 `ViewSet` и JWT-аутентификация

Для работы с задачами используется класс `TaskViewSet`, наследующий `ModelViewSet`. Переопределён метод `get_queryset`, который возвращает задачи, назначенные текущему пользователю или созданные им. Это обеспечивает изоляцию данных между пользователями.

`ViewSet` для работы с задачами (`backend/tasks/views.py`)

```
class TaskViewSet(viewsets.ModelViewSet):
    queryset = Task.objects.all()
    serializer_class = TaskSerializer
    permission_classes = [permissions.IsAuthenticated]

    def get_queryset(self):
        user = self.request.user
        return Task.objects.filter(
            Q(assigned_to=user) | Q(created_by=user)
        ).distinct()
```

Реализованы дополнительные действия `bulk_move` и `bulk_delete` для массового перемещения и удаления задач в пределах одной колонки. Доступ к API защищён с помощью `permissions.IsAuthenticated`, что гарантирует обработку запросов только от авторизованных пользователей.

2.5 Разработка клиентской части и вёрстки (Frontend)

2.5.1 Технологический стек frontend

Клиентская часть разработана на React 18 с использованием TypeScript. В качестве сборщика используется Vite. Для управления состоянием и серверными запросами используются библиотеки Zustand и TanStack Query. Для стилизации используется Tailwind CSS v4.

React позволяет создавать переиспользуемые компоненты и эффективно обновлять интерфейс при изменении данных. TypeScript обеспечивает проверку типов на этапе сборки, что снижает количество ошибок. Vite обеспечивает быструю сборку и горячую перезагрузку модулей в режиме разработки.

2.5.2 Реализация Kanban-доски

Главная страница приложения представляет собой Kanban-доску с тремя колонками: «К выполнению», «В работе» и «Готово». Задачи загружаются с сервера и группируются по статусам. Перемещение задач между колонками выполняется с помощью HTML5 Drag and Drop API. При сбросе задачи в другую колонку отправляется PATCH-запрос на обновление статуса.

Загрузка задач и обновление статуса (frontend/src/pages/TaskBoard.tsx)

```
const { data: tasks, isLoading } = useQuery({
  queryKey: ['tasks'],
  queryFn: async () => {
    const res = await api.get<Task[]>('tasks/');
    return res.data;
  },
});

const updateStatusMutation = useMutation({
  mutationFn: ({ id, status }: { id: number; status: string })
=>
  api.patch(`tasks/${id}/`, { status }),
  onSuccess: () => queryClient.invalidateQueries({ queryKey:
['tasks'] }),
});
```

Каждая карточка задачи отображает название, приоритет, идентификатор, описание, исполнителя и срок выполнения. Для задач с приближающимся дедлайном реализована подсветка. Доступны операции редактирования, удаления и массового перемещения.

2.5.3 Реализация календаря задач

Раздел «Календарь» реализован на чистом React без дополнительных библиотек. Календарь отображает задачи в соответствии с их сроком выполнения. Клик по дню открывает форму создания задачи с предзаполненной датой, а клик по задаче — форму редактирования.

Отображение задач в календаре (frontend/src/pages/CalendarPage.tsx)

```
<div className="grid grid-cols-7 gap-2 flex-1">
  {calendarDays.map(({ date, isCurrentMonth }, idx) => {
    const isoDate = toISODate(date);
    const dayTasks = tasks?.filter((t) =>
      getDatePart(t.due_date) === isoDate
    ) || [];
    return (
      <div key={idx} onClick={() => openCreateForm(date)}>
        <span>{date.getDate()}</span>
        {dayTasks.map((task) => (
          <div key={task.id}
            onClick={(e) => openEditForm(task, e)}>
            {task.title}
          </div>
        ))}
      </div>
    );
  })}
</div>
```

2.5.4 Темы оформления и адаптивность

Приложение поддерживает светлую и тёмную темы оформления. Переключение темы реализовано через React Context и сохраняется в localStorage браузера. В тёмной теме используется тёмно-синяя цветовая гамма с неоновыми акцентами, в светлой — светлые оттенки с индиго акцентами.

Интерфейс адаптирован для работы на различных устройствах. Боковая панель навигации скрывается на мобильных устройствах, а основной контент корректно отображается на экранах разного размера благодаря использованию CSS-Grid и Flexbox.

2.6 Автоматизация процессов разработки и развертывания (CI/CD)

2.6.1 Docker-контейнеризация

Для автоматизации развёртывания приложения используется Docker и Docker Compose. Backend и frontend собираются в отдельные контейнеры. Для обработки статических файлов и проксирования запросов используется Nginx.

Docker позволяет упаковать приложение со всеми зависимостями в изолированный контейнер, что обеспечивает одинаковое окружение на этапах разработки, тестирования и эксплуатации. Docker Compose позволяет описать и запустить несколько связанных контейнеров одной командой.

Dockerfile backend-контейнера описывает сборку образа на основе python:3.10-slim, установку зависимостей из requirements.txt и запуск сервера Gunicorn.

Dockerfile для backend (backend/Dockerfile)

```
FROM python:3.10-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["sh", "-c", "python manage.py migrate && gunicorn
config.wsgi:application --bind 0.0.0.0:8000"]
```

2.6.2 Процесс развёртывания

Файл docker-compose.yml описывает взаимодействие сервисов backend и frontend, общие тома для медиафайлов и порты для доступа к приложению. Запуск приложения выполняется командой `docker compose up -d --build`.

Конфигурация Docker Compose (docker-compose.yml)


```

services:
  backend:
    build:
      context: ./backend
      command: sh -c "python manage.py migrate && python manage.py
runserver 0.0.0.0:8000"
    volumes:
      - ./backend:/app
      - media_data:/app/media
    ports:
      - "8000:8000"
  frontend:
    build:
      context: ./frontend
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /etc/letsencrypt:/etc/letsencrypt:ro
      - media_data:/media
    depends_on:
      - backend

```

В production-окружении frontend обслуживается Nginx, который проксирует API-запросы к backend и раздаёт статические файлы. Для защиты соединения используются SSL-сертификаты Let's Encrypt. Система контроля версий Git используется для хранения исходного кода проекта.

2.7 Тестирование и верификация работоспособности

2.7.1 Методика тестирования

Тестирование проводилось методом функционального тестирования. Проверялись основные сценарии работы приложения: регистрация и авторизация пользователя, создание задачи, перемещение задач между колонками, редактирование и удаление задач, назначение исполнителя, установка срока выполнения, переключение тем оформления, работа календаря.

Функциональное тестирование позволяет проверить соответствие реализованного функционала требованиям технического задания. Каждый сценарий выполнялся вручную: создавалась тестовая учётная запись, формировались задачи с разными приоритетами и сроками, проверялось перемещение между статусами и отображение в календаре.

2.7.2 Результаты функционального тестирования

В результате тестирования установлено, что все заявленные функции работают корректно. Пользователи могут регистрироваться, создавать задачи, перемещать их между статусами, назначать приоритеты и сроки выполнения, выбирать исполнителей. Приложение корректно отображается в светлой и тёмной темах. Развёртывание в Docker-контейнерах выполнено успешно.

В ходе тестирования не выявлено критических ошибок. Все API-запросы возвращают ожидаемые данные, аутентификация работает корректно, интерфейс отзывчивый и понятный. Приложение развёрнуто на VPS-сервере и доступно по доменному имени `taskmanager2026.ru`.

2.8 Описание пользовательского интерфейса

2.8.1 Страница авторизации и регистрации

Первый экран, с которым сталкивается пользователь, — это страница авторизации. На ней расположены поля для ввода логина и пароля, кнопка входа и ссылка для перехода к форме регистрации. Интерфейс выполнен в лаконичном стиле и адаптирован как для светлой, так и для тёмной темы оформления.

Форма регистрации запрашивает у пользователя имя, адрес электронной почты, логин и пароль. Перед отправкой данных на сервер выполняется клиентская валидация: проверяется заполнение обязательных полей, длина

пароля и совпадение введенных паролей. В случае ошибок пользователь получает понятные сообщения под соответствующими полями.

После успешной авторизации пользователь попадает на главную страницу приложения. Access-токен сохраняется в памяти приложения и используется для всех последующих запросов к защищенным ресурсам API. Если срок действия токена истекает, приложение автоматически обновляет его с помощью refresh-токена.

2.8.2 Главная страница и Kanban-доска

Главная страница приложения представляет собой Kanban-доску, разделенную на несколько колонок, соответствующих статусам задач. В стандартной конфигурации используются колонки «Новые», «В работе», «На проверке» и «Выполнены». Каждая задача отображается в виде карточки, содержащей заголовок, описание, приоритет, срок выполнения и имя исполнителя.

Пользователь может перемещать задачи между колонками с помощью drag-and-drop. При изменении статуса приложение отправляет PATCH-запрос к API, и состояние доски обновляется. Также реализованы массовые операции: перемещение всех задач колонки в другой статус и удаление всех задач в колонке. Это упрощает управление большим количеством записей.

Над доской расположена панель инструментов, содержащая кнопку создания новой задачи, переключатель темы оформления, кнопку перехода к календарю и меню пользователя. Боковая панель навигации содержит ссылки на основные разделы приложения и отображает краткую информацию о текущем пользователе.

2.8.3 Страница календаря и настройки профиля

Страница календаря позволяет просматривать задачи в привязке к датам выполнения. Каждая задача отображается в ячейке соответствующего дня. При клике на задачу открывается модальное окно с подробной информацией и возможностью редактирования. Календарь поддерживает переключение между месяцами и адаптируется под размер экрана.

В настройках профиля пользователь может изменить своё отображаемое имя, адрес электронной почты и пароль. Интерфейс настроек выполнен в едином стиле с остальными страницами приложения. Все изменения профиля сопровождаются валидацией данных и уведомлениями об успешном сохранении или возникших ошибках.

2.9 Руководство пользователя

2.9.1 Регистрация и вход в систему

Для начала работы с приложением пользователь должен зарегистрироваться. Для этого необходимо перейти по ссылке «Регистрация» на странице авторизации, заполнить все обязательные поля формы и нажать кнопку «Зарегистрироваться». После успешной регистрации пользователь может войти в систему, используя указанные логин и пароль.

Вход в систему выполняется на странице авторизации. Пользователь вводит логин и пароль, после чего приложение отправляет запрос к API. При успешной аутентификации пользователь получает access-токен и refresh-токен, которые используются для доступа к защищённым ресурсам. В случае неверных учётных данных выводится сообщение об ошибке.

2.9.2 Создание и редактирование задач

Для создания новой задачи необходимо нажать кнопку «Создать задачу» на главной странице. В открывшемся модальном окне пользователь заполняет название задачи, описание, выбирает приоритет, устанавливает срок выполнения и назначает исполнителя. После нажатия кнопки «Сохранить» задача появляется в колонке «Новые».

Редактирование задачи выполняется путём открытия карточки задачи. Пользователь может изменить любое поле: название, описание, приоритет, срок выполнения, исполнителя и статус. Все изменения сохраняются на сервере и отображаются на Kanban-доске и в календаре. Удаление задачи выполняется через соответствующую кнопку в карточке задачи.

2.9.3 Управление статусами задач

Изменение статуса задачи выполняется перетаскиванием карточки между колонками Kanban-доски. При этом автоматически отправляется запрос к API, и статус задачи обновляется в базе данных. Такой подход позволяет наглядно отслеживать прогресс выполнения работы.

Для повышения удобства работы с большим количеством задач реализованы массовые операции. Пользователь может переместить все задачи из одной колонки в другую или удалить все задачи в выбранной колонке. Перед выполнением массового удаления выводится подтверждающее диалоговое окно, предотвращающее случайную потерю данных.

2.10 Оценка качества и перспективы развития

2.10.1 Оценка качества реализации

Разработанное приложение соответствует поставленным функциональным и нефункциональным требованиям. Реализованы все

заявленные возможности: регистрация и авторизация пользователей, создание и редактирование задач, управление статусами через Kanban-доску, просмотр задач в календаре, поддержка тёмной и светлой тем, массовые операции с задачами и развёртывание в Docker.

Код приложения структурирован и разделён на логические модули. Backend-часть использует сериализаторы для валидации данных, ViewSet'ы для обработки запросов и систему разрешений для ограничения доступа. Frontend-часть построена на компонентах React, что упрощает поддержку и расширение интерфейса.

Производительность приложения достаточна для работы с несколькими сотнями задач одновременно. Время отклика API не превышает нескольких сотен миллисекунд, а интерфейс остаётся отзывчивым при типичных пользовательских операциях. Контейнеризация обеспечивает стабильность окружения и упрощает процесс развёртывания.

2.10.2 Перспективы дальнейшего развития

В дальнейшем планируется расширение функционала приложения. Первым направлением является добавление системы уведомлений о приближающихся сроках выполнения задач. Уведомления могут отображаться в интерфейсе приложения и отправляться по электронной почте, что позволит пользователям не пропускать важные дедлайны.

Вторым направлением является улучшение возможностей фильтрации и поиска задач. Планируется добавить фильтры по приоритету, статусу, исполнителю и диапазону дат, а также полнотекстовый поиск по названию и описанию задач. Это повысит удобство работы с большим количеством записей.

Также планируется внедрение ролевой модели с разделением прав доступа. Роли администратора, менеджера и исполнителя позволят гибко

управлять доступом к задачам и функциям приложения. Дополнительно рассматривается возможность экспорта списка задач в форматы CSV и PDF, что упростит формирование отчётов.

Для повышения надёжности планируется добавить автоматизированные unit-тесты для backend-части и интеграционные тесты для проверки взаимодействия компонентов системы. В перспективе также возможна настройка непрерывной интеграции и доставки (CI/CD), которая будет автоматически запускать тесты и развёртывать приложение при обновлении кода в репозитории.

2.11 Документирование API и примеры использования

В процессе разработки backend-части было сформировано REST API, которое используется клиентской частью для обмена данными. API построено по принципам REST: каждый ресурс имеет свой URI, а операции над ресурсами выполняются с помощью стандартных HTTP-методов. Для документирования API используются сериализаторы Django REST Framework, которые описывают поля запросов и ответов.

Основные endpoints приложения приведены в таблице. Все запросы к защищённым ресурсам требуют передачи access-токена в заголовке Authorization. Токен получается после успешной авторизации по адресу /api/token/.

Ресурс /api/tasks/ поддерживает операции создания (POST), чтения списка (GET), обновления (PATCH) и удаления (DELETE) отдельных задач. Маршрутизация формируется автоматически с помощью DefaultRouter Django REST Framework. Дополнительные действия bulk_move и bulk_delete доступны по адресам /api/tasks/bulk_move/ и /api/tasks/bulk_delete/.

Сериализатор Task (backend/tasks/serializers.py)

```

class TaskSerializer(serializers.ModelSerializer):
    created_by = UserSerializer(read_only=True)
    assigned_to = UserSerializer(read_only=True)

    class Meta:
        model = Task
        fields = [
            'id', 'title', 'description', 'status',
            'priority', 'due_date', 'assigned_to', 'created_by',
            'created_at',
        ]
        read_only_fields = ['id', 'created_at', 'created_by']

```

Сериализатор TaskSerializer преобразует объекты модели Task в JSON и выполняет обратное преобразование при создании и обновлении задач. Поля created_by и assigned_to представлены вложенными сериализаторами пользователя, что позволяет сразу получить расширенную информацию о создателе и исполнителе задачи.

Ниже приведён пример запроса на создание задачи и соответствующего ответа сервера. В заголовке запроса передаётся JWT access-токен, полученный при авторизации. В теле запроса указываются название задачи, описание, статус, приоритет, срок выполнения и идентификатор исполнителя.

Пример POST-запроса на создание задачи

```

POST /api/tasks/ HTTP/1.1
Host: taskmanager2026.ru
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
Content-Type: application/json

{
    "title": "Подготовить отчёт",
    "description": "Сформировать ежемесячный отчёт по задачам",
    "status": "new",
    "priority": "high",
    "due_date": "2026-06-30T12:00:00Z",
    "assigned_to": 2
}

```

Пример ответа сервера при создании задачи

HTTP/1.1 201 Created

Content-Type: application/json

```
{
  "id": 42,
  "title": "Подготовить отчёт",
  "description": "Сформировать ежемесячный отчёт по задачам",
  "status": "new",
  "priority": "high",
  "due_date": "2026-06-30T12:00:00Z",
  "assigned_to": {
    "id": 2,
    "username": "petrov"
  },
  "created_by": {
    "id": 1,
    "username": "ivanov"
  },
  "created_at": "2026-06-22T09:15:00Z"
}
```

Для обновления статуса задачи используется PATCH-запрос. В теле запроса передаются только изменяемые поля, что соответствует семантике метода PATCH. После успешного обновления клиентская часть автоматически обновляет данные на Kanban-доске и в календаре благодаря использованию TanStack Query.

Для массового перемещения задач между колонками реализован пользовательский action `bulk_move`. Он принимает список идентификаторов задач и целевой статус, после чего обновляет статус всех указанных задач, доступных текущему пользователю. Подобным образом работает action `bulk_delete`, который удаляет выбранные задачи.

2.12 Безопасность и защита данных в приложении

Безопасность приложения обеспечивается на нескольких уровнях: аутентификация и авторизация запросов, защита от межсайтовых атак, валидация входных данных и использование защищённого канала передачи данных. При проектировании архитектуры особое внимание уделялось

минимизации рисков, связанных с несанкционированным доступом и некорректной обработкой данных.

Аутентификация пользователей реализована с помощью JWT-токенов. Библиотека `django-rest-framework-simplejwt` предоставляет endpoints для получения и обновления токенов. Access-токен имеет короткое время жизни, что ограничивает возможности злоумышленника в случае его перехвата. Refresh-токен хранится в `localStorage` браузера и используется для получения нового access-токена без повторного ввода пароля.

Для защиты от подделки межсайтовых запросов и обеспечения кросс-доменного взаимодействия в настройках Django подключено приложение `django-cors-headers`. В production-окружении список разрешённых источников ограничивается доменом frontend-приложения. Это предотвращает выполнение запросов к API со сторонних сайтов.

Настройки аутентификации и CORS (`backend/config/settings.py`)

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [

'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
}

CORS_ALLOWED_ORIGINS = [
    'https://taskmanager2026.ru',
]
```

Для защиты передаваемых данных используется протокол HTTPS. SSL-сертификаты получены в удостоверяющем центре Let's Encrypt и автоматически обновляются. Nginx выполняет перенаправление HTTP-

запросов на HTTPS и добавляет заголовки безопасности, ограничивающие выполнение встроенного содержимого и защищающие от ряда атак.

Конфигурация Nginx с SSL (nginx/nginx.conf)

```
server {
    listen 80;
    server_name taskmanager2026.ru;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name taskmanager2026.ru;

    ssl_certificate
/etc/letsencrypt/live/taskmanager2026.ru/fullchain.pem;
    ssl_certificate_key
/etc/letsencrypt/live/taskmanager2026.ru/privkey.pem;

    location / {
        root /var/www/frontend;
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Права доступа к задачам ограничены на уровне queryset. Пользователь видит только задачи, созданные им или назначенные ему. Это исключает возможность просмотра и изменения чужих задач через прямой вызов API. Дополнительная проверка прав выполняется в сериализаторе и ViewSet.

Все входные данные проходят валидацию на клиенте и на сервере. На клиенте проверяется заполнение обязательных полей и корректность форматов дат. На сервере сериализаторы Django REST Framework выполняют повторную проверку, а ORM Django защищает от SQL-инъекций за счёт использования

параметризованных запросов. Комбинация этих мер обеспечивает устойчивость приложения к распространённым угрозам.

Заключение

В ходе выполнения курсовой работы было разработано полноценное веб-приложение для управления задачами. Приложение имеет клиентскую и серверную части, систему аутентификации на основе JWT, реализованную Kanban-доску с drag-and-drop, поддержку тёмной и светлой тем, календарь задач и возможность развёртывания в Docker.

Серверная часть приложения реализована на языке Python с использованием фреймворка Django и Django REST Framework. Для хранения данных используется реляционная база данных SQLite. API защищено с помощью JWT-токенов, а доступ к задачам ограничен создателями и исполнителями. Реализованы массовые операции с задачами, что упрощает управление большим количеством записей.

Клиентская часть разработана на React с использованием TypeScript. Интерфейс представляет собой адаптивное одностраничное приложение с боковой панелью навигации, Kanban-доской и календарём. Для стилизации используется Tailwind CSS, реализована поддержка светлой и тёмной тем оформления. Взаимодействие с сервером выполняется через библиотеку Axios, а управление состоянием и кешированием данных реализовано с помощью TanStack Query.

Полученный результат соответствует поставленным требованиям и может быть использован для управления личными и командными задачами. Приложение развёрнуто на VPS-сервере и доступно по доменному имени taskmanager2026.ru.

В ходе работы были приобретены и закреплены практические навыки проектирования REST API, разработки клиент-серверных приложений, работы с системой контроля версий Git, контейнеризации приложений с помощью Docker и развёртывания на удалённом сервере. Особое внимание уделялось

обеспечению безопасности данных пользователей и корректной работе аутентификации.

Разработанное приложение демонстрирует применение современных подходов к веб-разработке: разделение ответственности между backend и frontend, использование JWT-токенов для аутентификации, применение утилитарных CSS-фреймворков для быстрой вёрстки и использование контейнеризации для упрощения развёртывания.

В дальнейшем планируется добавить функционал уведомлений о приближающихся сроках выполнения задач, расширить возможности фильтрации и поиска, добавить ролевую модель с правами администратора и менеджера, а также реализовать экспорт списка задач в формате CSV или PDF. Также планируется добавить автоматизированные unit-тесты для backend-части и интеграционные тесты для проверки взаимодействия компонентов системы.

Список использованных источников

1. ГОСТ Р 7.0.100-2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — Москва: Стандартинформ, 2018. — 132 с.
2. ГОСТ Р 7.0.108-2022. Система стандартов по информации, библиотечному и издательскому делу. Библиографические ссылки на электронные документы, размещенные в информационно-телекоммуникационных сетях. Общие требования к составлению. — Москва: Российский институт стандартизации, 2022. — 23 с.
3. ГОСТ Р ИСО 9241-210-2016. Эргономика взаимодействия человек-система. Часть 210. Человеко-ориентированное проектирование интерактивных систем. — Москва: Стандартинформ, 2016. — 32 с.
4. Федеральный закон от 27.07.2006 № 149-ФЗ (ред. от 08.08.2024) «Об информации, информационных технологиях и о защите информации». — Москва, 2024.
5. Федеральный закон от 27.07.2006 № 152-ФЗ (ред. от 21.07.2024) «О персональных данных». — Москва, 2024.
6. Федеральный закон от 26.07.2017 № 187-ФЗ «О безопасности критической информационной инфраструктуры Российской Федерации». — Москва, 2017.
7. СанПиН 2.2.2/2.4.1340-03. Гигиенические требования к персональным электронно-вычислительным машинам и организации работы. — Москва, 2016.

8. Лутц, М. Изучаем Python. — 5-е изд. — Санкт-Петербург: Диалектика, 2023. — 1360 с.
9. Дронов, В.А. Django: практика создания Web-сайтов на Python. — Санкт-Петербург: БХВ-Петербург, 2022. — 704 с.
10. Добридняк, С.П. Разработка веб-приложений на Django. — Москва: ДМК Пресс, 2023. — 352 с.
11. Орлов, С.А. Технологии разработки программного обеспечения. — 6-е изд. — Санкт-Петербург: Питер, 2023. — 672 с.
12. Коптелов, А.К. Проектирование информационных систем. — Москва: Юрайт, 2024. — 287 с.
13. Голован, С.В. REST API: проектирование и разработка. — Москва: Альпина Паблишер, 2023. — 248 с.
14. Ручкин, В.Н. WEB-программирование на Python. — Москва: Инфра-М, 2024. — 319 с.
15. Самойлов, М.В. Основы программной инженерии. — Томск: Издательство Томского университета, 2023. — 198 с.
16. Percival, H. Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices. — Sebastopol, CA: O'Reilly Media, 2020. — 300 p.
17. Ramalho, L. Fluent Python: Clear, Concise, and Effective Programming. — 2nd ed. — Sebastopol, CA: O'Reilly Media, 2022. — 1012 p.
18. Django Software Foundation. Django documentation [Электронный ресурс]. — URL: <https://docs.djangoproject.com/en/5.0/>.

19. Django REST Framework. Django REST framework documentation [Электронный ресурс]. — URL: <https://www.django-rest-framework.org/>.
20. React. React documentation [Электронный ресурс]. — URL: <https://react.dev/>.
21. TypeScript. TypeScript documentation [Электронный ресурс]. — URL: <https://www.typescriptlang.org/>.
22. Tailwind CSS. Tailwind CSS documentation [Электронный ресурс]. — URL: <https://tailwindcss.com/>.
23. Vite. Vite documentation [Электронный ресурс]. — URL: <https://vitejs.dev/>.
24. Docker. Docker documentation [Электронный ресурс]. — URL: <https://docs.docker.com/>.
25. GitHub. Репозиторий проекта Task Manager [Электронный ресурс]. — URL: <https://github.com/kama326/task-manager.git>.